

Image Segmentation using EM-Algorithm

Statistical Methods II Project

A.N.S.M. Tamimuddin, Anunoy Chal, Inesh Chattopadhyay,
Ankit Bhar, Aryama Ghosh

May 12, 2024

Abstract

We want to explore one possible area of application for the Expectation-Maximization algorithm for computing Maximum Likelihood Estimate. Our point of interest is the segmentation of an image into possible objects under the hypothesis of a Gaussian mixture model.

Background

We work under the hypothesis that the colour components of an image follow a Gaussian mixture model, with unknown weights assigned to the Gaussian densities modelling the distribution of the colours. By de-clustering this mixture model, we aim to identify and separate different regions, possibly denoting distinct real-world objects, in a single image, which can be applied to multiple practical settings such as reading a bar code, et cetera.

Theory & Convergence

In statistics, an expectation-maximization (EM) algorithm is an iterative process to find local maximum likelihood estimates of parameters in statistical models which depend on unobserved latent variables. The EM algorithm alternates between an E -step which involves computing the expected value of the log-likelihood evaluated using the current estimates of the parameters and an M -step which involves maximizing the expectation derived from the E -step on the parameter space. Our work refers to a Gaussian mixture model which is a specific case of this algorithm where we assume that each data point has the corresponding unobserved factor of belonging to a mixture component of different Gaussian densities with unknown weights.

We are given some data $\mathbf{X}$ where we assume some unobserved latent data $\mathbf{Z}$ and an unknown parameter θ . Let us denote the likelihood function by $L(\theta; \mathbf{X}, \mathbf{Z}) = P(\mathbf{X}, \mathbf{Z} | \theta)$. Then, finding the maximum likelihood estimate corresponds to maximizing the following

$$L(\theta; \mathbf{X}) = p(\mathbf{X} | \theta) = \int p(\mathbf{X}, \mathbf{Z} | \theta) d\mathbf{Z} = \int p(\mathbf{X} | \mathbf{Z}, \theta) p(\mathbf{Z} | \theta) d\mathbf{Z}$$

This quantity, however, is often intractable since $\mathbf{Z}$ is unobserved and its distribution is unknown before attaining θ . What we do is define the quantity Q in the E -step as follows

$$Q(\boldsymbol{\theta} \mid \boldsymbol{\theta}^{(t)}) = \mathbb{E}_{\mathbf{Z}} [\log p(\mathbf{X}, \mathbf{Z} \mid \boldsymbol{\theta})],$$

that is, the expected value of the log-likelihood of $\boldsymbol{\theta}$ with respect to the conditional distribution of $\mathbf{Z}$ given $\mathbf{X}$ and $\boldsymbol{\theta}^{(t)}$. In the M -step, we find $\boldsymbol{\theta}^{(t+1)} \in \Theta$ such that $Q(\boldsymbol{\theta}^{(t+1)} \mid \boldsymbol{\theta}^{(t)})$ is maximum. Then we use this new $\boldsymbol{\theta}^{(t+1)}$ and proceed with the E -Step. We repeat this process until convergence.

As to the convergence of this algorithm, one needs certain regularity conditions in place. It follows due to Jensen's inequality that the sequence of log-likelihoods computed in the process forms a monotonically non-decreasing sequence.

The complete log-likelihood function is as follows: $L_c(\boldsymbol{\theta}) = \ln p(\mathbf{X}, \mathbf{Z} \mid \boldsymbol{\theta}) = L(\boldsymbol{\theta}) + \ln p(\mathbf{Z} \mid \mathbf{X}, \boldsymbol{\theta})$, where X and Z are the observed and unobserved data respectively. Taking expectation on both sides with respect to the conditional distribution of $\mathbf{Z} \mid \mathbf{X}$, we get $Q(\boldsymbol{\theta}, \boldsymbol{\theta}^{(t)}) = \ln(\boldsymbol{\theta}) + H(\boldsymbol{\theta}, \boldsymbol{\theta}^{(t)})$ where $H(\boldsymbol{\theta}, \boldsymbol{\theta}^{(t)}) = \mathbb{E}[\ln p(z \mid x, \boldsymbol{\theta}) \mid y]$. Due to Jensen's inequality, we have $H(\boldsymbol{\theta}^{(t+1)}; \boldsymbol{\theta}^{(t)}) \leq H(\boldsymbol{\theta}^{(t)}; \boldsymbol{\theta}^{(t)})$ and therefore,

$$\begin{aligned} \ln L(\boldsymbol{\theta}^{(t+1)}) - \ln L(\boldsymbol{\theta}^{(t)}) &= (Q(\boldsymbol{\theta}^{(t+1)}; \boldsymbol{\theta}^{(t)}) - Q(\boldsymbol{\theta}^{(t)}; \boldsymbol{\theta}^{(t)})) - (H(\boldsymbol{\theta}^{(t+1)}; \boldsymbol{\theta}^{(t)}) - H(\boldsymbol{\theta}^{(t)}; \boldsymbol{\theta}^{(t)})) \\ &\geq (Q(\boldsymbol{\theta}^{(t+1)}; \boldsymbol{\theta}^{(t)}) - Q(\boldsymbol{\theta}^{(t)}; \boldsymbol{\theta}^{(t)})) \geq 0, \end{aligned}$$

where the last inequality follows from the definition of $\boldsymbol{\theta}^{(t+1)}$. In other words, the sequence of iterates $\{\boldsymbol{\theta}^{(k)}\}$ forms a monotonically non-decreasing sequence of log-likelihoods which converges to some L^* if it is bounded. Now questions naturally arise as to the conditions under which L^* corresponds to a stationary value and when this stationary value is at least a local maximum if not a global maximum. In our case, since we can show that the likelihood function is a log-concave function, we are guaranteed the existence of a unique local maxima.

K-means clustering is also an iterative process which aims to segregate a data into mutually exclusive and exhaustive sets (called 'clusters'), where each observation in the data is assigned to the cluster with nearest mean (termed 'cluster means' or 'centroids'), 'nearness' being measured in terms of Euclidean distance between two points. The standard K-means clustering algorithm starts off with an initial guess of cluster means and involves two steps in each iteration: Assignment step, which involves assigning every data point to the nearest cluster mean, and an Update step, which recalculates the centroids and improves those, which would be utilized for the next iteration.

Suppose we have a set of d -dimensional observations $\{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n\}$ and our objective is to partition the set into k ($< n$) many sets $\mathbf{S} = \{S_1, \dots, S_k\}$ so as to minimise 'within-cluster' variance, which

formulating mathematically translates to:

$$\arg \min_{\mathbf{S}} \sum_{i=1}^k \sum_{\mathbf{x} \in S_i} \|\mathbf{x} - \boldsymbol{\mu}_i\|^2$$

Here, $\boldsymbol{\mu}_i$ is the centroid of points in S_i , given by $\boldsymbol{\mu}_i = \frac{1}{|S_i|} \sum_{\mathbf{x} \in S_i} \mathbf{x}$, where $|S_i|$ denotes the size of the set S_i and $\|\cdot\|$ is the usual L_2 norm.

The standard k-means clustering algorithm goes like this: we initiate the process with our initial guess of k means $\mathbf{m}_1^{(1)}, \dots, \mathbf{m}_k^{(1)}$, and the t -th iteration step involves:

Assignment Step: In this step, each observation is assigned to its nearest mean (and the corresponding cluster, each data point is assigned to exactly one cluster) among $\mathbf{m}_1^{(t)}, \dots, \mathbf{m}_k^{(t)}$ and reform the clusters as

$$S_i^{(t)} = \left\{ \mathbf{x} : \left\| \mathbf{x} - \mathbf{m}_i^{(t)} \right\|^2 \leq \left\| \mathbf{x} - \mathbf{m}_j^{(t)} \right\|^2 \quad \forall j, 1 \leq j \leq k \right\}$$

Update Step: This is the step to recalculate and update centroids,

$$\mathbf{m}_i^{(t+1)} = \frac{1}{|S_i^{(t)}|} \sum_{\mathbf{x} \in S_i^{(t)}} \mathbf{x}$$

The process is stopped when the change in assignments is insignificant.

Gaussian Mixture Model in Brief: In statistics, a mixture model is a probabilistic model for representing the presence of sub-populations within an overall population, without requiring that an observed data set should identify the sub-population to which an individual observation belongs. Here, we consider only the Gaussian family. In a typical setting, we want to estimate the parameters $\boldsymbol{\mu}_i, \boldsymbol{\Sigma}_i$ and the mixture weights π_i 's in the mixture distribution function given by

$$p(\mathbf{x}) = \sum_{i=1}^K \pi_i \mathcal{N}_k(\mathbf{x} | \boldsymbol{\mu}_i, \boldsymbol{\Sigma}_i).$$

where K is the number of mixture components, or simply, clusters.

Now, we discuss how the EM algorithm applies to this case of Gaussian Mixture Models.

Setup:

We have an i.i.d sample $\{\mathbf{X}_1, \mathbf{X}_2, \dots, \mathbf{X}_n\}$ from a mixture of Gaussian ditribution with density,

$p(\mathbf{x}) = \sum_{i=1}^K \pi_i \mathcal{N}(\mathbf{x} | \boldsymbol{\mu}_i, \boldsymbol{\Sigma}_i)$. We wish to estimate the parameters given by $\boldsymbol{\theta} = \{\boldsymbol{\mu}_i, \boldsymbol{\Sigma}_i, \pi_i : 1 \leq i \leq K\}$ for which the log likelihood of the observed data $\mathbf{X}$ is maximized.

Log Likelihood of the observed data:

$$\begin{aligned}
l(\mathbf{X}|\boldsymbol{\theta}) &= \log(P(X_i = x_i, 1 \leq i \leq n \mid \boldsymbol{\theta})) \\
&= \sum_i \log \sum_k P(Z_i = k|\boldsymbol{\theta})P(X_i = x_i|Z_i = k, \boldsymbol{\theta}) \\
&= \sum_i \log \sum_k \boldsymbol{\pi}_k \mathcal{N}(x_i|\mu_k, \Sigma_k).
\end{aligned}$$

Here, the i -th data point $\mathbf{x}_i$ has a corresponding latent random variable Z_i representing the mixture component that it came from. So, each $\mathbf{Z}_i \sim \text{Multinomial}(\boldsymbol{\pi})$, where $\boldsymbol{\pi} = (\boldsymbol{\pi}_1, \boldsymbol{\pi}_2, \dots, \boldsymbol{\pi}_K)$. Since the observed data is i.i.d, the $\mathbf{Z}_i$'s are all independent. Note that

$$\begin{aligned}
\sum_i \log(p(x_i, z_i)) &= \sum_i [\log(p(x_i|z_i)) + \log p(z_i)] \\
&= \sum_i [\log(\mathcal{N}(x_i, \mu_{z_i}, \Sigma_{z_i})) + \log(\pi_{z_i})] \\
&= \sum_i \sum_j \mathbb{1}\{z_i = j\} [\log(\mathcal{N}(x_i, \mu_j, \Sigma_j)) + \log(\pi_j)].
\end{aligned}$$

For a $\boldsymbol{\theta} \in \Theta$, the conditional expectation of the log-likelihood of the complete data given the current $\boldsymbol{\theta}^{(t)}$ and observed data $\mathbf{X}$ is the following:

$$Q(\boldsymbol{\theta}; \boldsymbol{\theta}^{(t)}) = \mathbb{E}(l(\boldsymbol{\theta}; \mathbf{X}, \mathbf{Z})|\mathbf{X}, \boldsymbol{\theta}^{(t)}) = \sum_{i,j} \underbrace{P(z_i = j \mid \mathbf{x}_j, \boldsymbol{\theta})}_{\text{call it } \tau_{ij}} \log \mathcal{N}(\mathbf{x}_i \mid \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j).$$

If we go by the notation $\sum_i \tau_{ij} = N_j$, this nudges us to assign the parameters as described by

$$\boldsymbol{\mu}_k = \frac{1}{N_k} \sum_{n=1}^N r_{nk} \mathbf{x}_n; \quad \boldsymbol{\Sigma}_k = \frac{1}{N_k} \sum_{n=1}^N r_{nk} (\mathbf{x}_n - \boldsymbol{\mu}_k)(\mathbf{x}_n - \boldsymbol{\mu}_k)^T; \quad \pi_k = \frac{N_k}{N}. \quad (1)$$

Data & Implementation

Link to the Codes

Here, we used usual images with real objects(i.e.not animation or any lineart) and hypothesized that the RGB color components in the image, came from a mixture of Gaussians, where each Gaussian possibly represents a segment of the image or, a possible object. Then declustering is performed. The code is entirely written in Python, using the libraries Numpy, Matplotlib and Scipy. Numpy functions are a lot faster than Python's built in functions, for being implemented in C. We used these wherever we could, to make the code efficient.

At the end, upon obtaining the responsibility matrix, we label the pixels accordingly and assign each pixel the centroid of the cluster that it belongs to and obtain the new image.

Now, for the obtained image, in order to reduce the number of pixels that are possibly somewhat mislabelled and thereby stand out as irregularities , we do the following.

1. Consider the $(h \times w)$ matrix $\mathbf{L}$, such that L_{ij} = label of the (i, j) -th pixel of the image.

2. For each pixel v at position (i, j) , consider the window centered at L_{ij} and let the mode of the values within this window be m .

Assign: $v = \text{centroid of the } m\text{-th cluster}$

Essentially, for a possibly mislabelled pixel, it shall, in a sense, stand out from the rest of the pixels in its neighborhood. So, we replace its value by the most occurring value in its neighbourhood. We take a window size of 3.

Finding the optimal number of clusters

The straightforward way of finding this out is to simply perform k-Means with different number of clusters and record, at each step, the value of the objective function that k-Means attempts to minimize. Then, plot a graph of these values against the number of clusters. The number of clusters at which the objective function sharply hits a low value and bringing in more clusters afterwards doesn't change the objective function by much, is then concluded to be the optimal number of clusters. The point at which this sharp hit occurs is called the 'elbow'. The optimal number is decided from looking at the graph. Now, while this may seem reasonable, not only in clustered datasets but this elbow is observed even in uniform datasets.

The objective function in K-Means is the following:

$$\arg \min_{\mathbf{S}} \sum_{i=1}^k \frac{1}{|S_i|} \sum_{\mathbf{x}, \mathbf{y} \in S_i} \|\mathbf{x} - \mathbf{y}\|^2 = \arg \min_{\mathbf{S}} \sum_{i=1}^k \sum_{\mathbf{x} \in S_i} \|\mathbf{x} - \boldsymbol{\mu}_i\|^2$$

That is, the within cluster sum of squares (WCSS). Now, for a data that's somewhat well clustered, at an optimal number of clusters, the within cluster sum of squares is minimized and the sum of squares between the clusters themselves is maximized. The latter sum is BCSS (between cluster sum of squares) and is given by the following:

$$\text{BCSS} = \sum_{i=1}^k n_i \|\mathbf{c}_i - \mathbf{c}\|^2$$

where n_i is the number of points in cluster C_i , $\mathbf{c}_i$ is the centroid of C_i , and $\mathbf{c}$ is the overall centroid of the data. BCSS measures how well the clusters are separated from each other (the higher the better).

VRC Criterion

So, instead of only the WCSS, we consider the CH-index, given by the following:

$$\text{CH} = \frac{\text{BCSS}/(k-1)}{\text{WCSS}/(n-k)}$$

The BCSS and WCSS are normalized by dividing by $k-1$ and $n-k$ respectively, so that they stay comparable across different number of clusters.

Heuristically speaking, for the optimal number of clusters, the BCSS shall be quite high owing to good separation of the clusters, and the WCSS shall be low. This should result in a high CH index. Now, we run this with different number of clusters, and in each case, we run K-Means 5 times and calculate the CH index each time and then record the highest CH index among these to be the value corresponding to that number of clusters. We run K-Means 5 times to ensure not getting any ‘strange’ value of the CH index resulting from strange initialisations. For each k , we perform 30 iterations of K-Means. The fit often gets better for higher number of clusters and so, we choose k accordingly to avoid overfitting. Also, here, we resized the images to a suitable order to curb the computational time and cost while retaining significant pixel information. (i.e. For a 400×266 image, we resize it to (200×133) and then proceed with CH.)

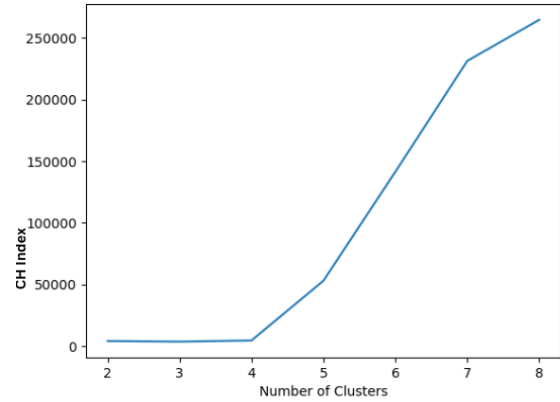
We perform the iterations for different values of k (until reasonable stability) before computing the within-cluster dispersion σ_k . Then we optimise σ_k . Although this makes heuristic sense, it is expensive. Graphically, it corresponds to finding the elbow of the ‘variance versus k ’ curve, hence the name.

Results:

The following are a few results from the segmentations.



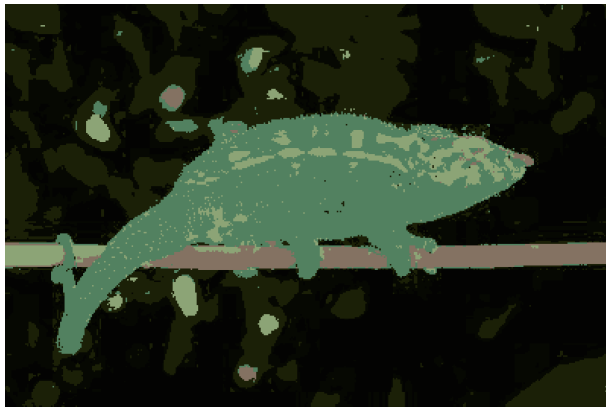
(a) Original Image



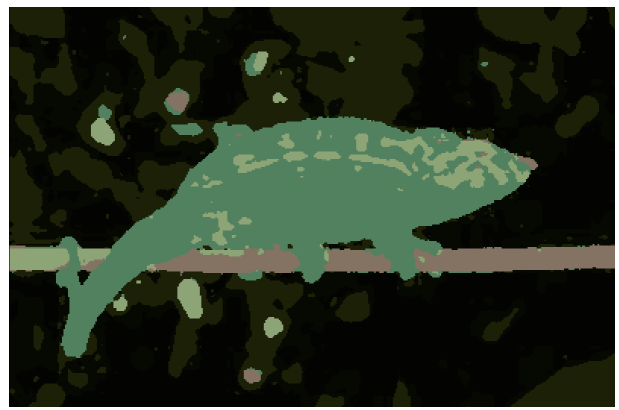
(b) Corresponding graph for the CH indices

Figure 1

We conduct segmentations with 5 and 6 clusters. We avoid opting for higher number of clusters to not run into overfitting and possibly noisy segmentations.

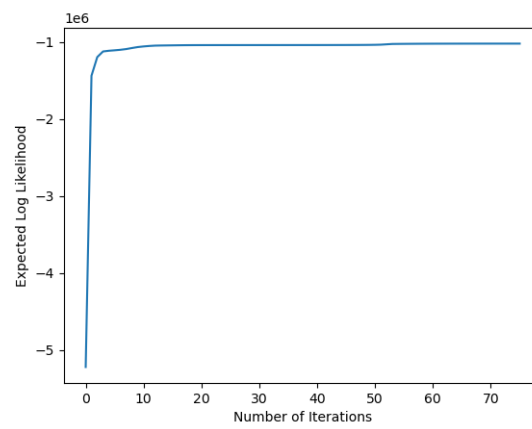


(a) 55 iterations of KMeans, 75 iterations of EM, 6 clusters



(b) 55 iterations of KMeans, 75 iterations of EM, 6 clusters, refined

Figure 2

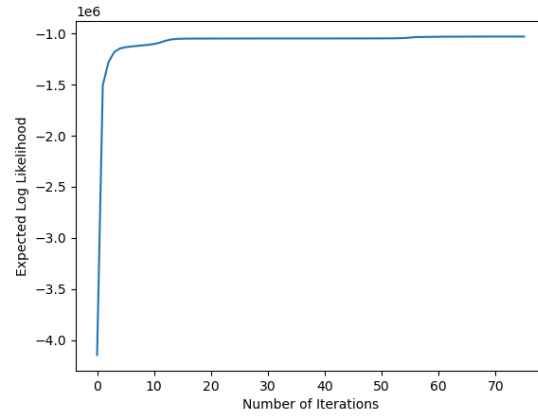


(a) 55 iterations of KMeans, 75 iterations of EM, 5 clusters



(b) Refined

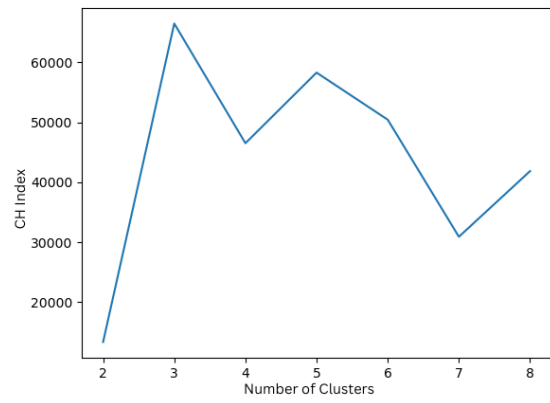
Figure 3



We may see that the refinement reduces quite some of the irregularities.



(a) Original Image



(b) Corresponding graph for the CH indices

Figure 4

We segment with 3 clusters.

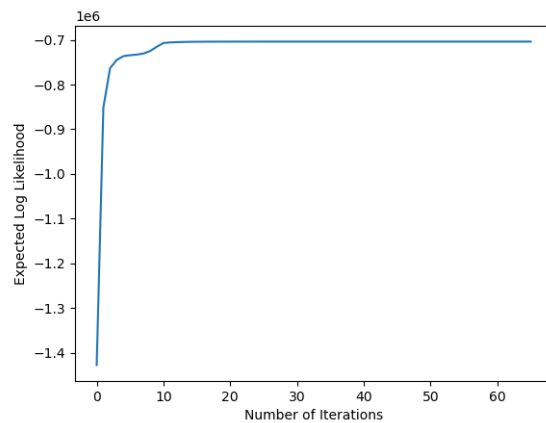


(a) 50 iterations of KMeans, 65 iterations of EM, 3 clusters

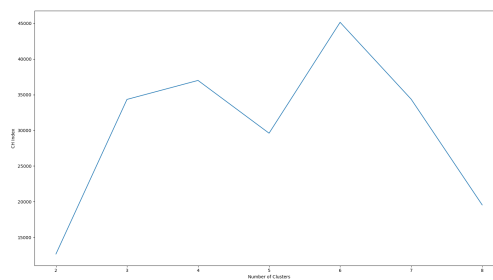


(b) Refined

Figure 5

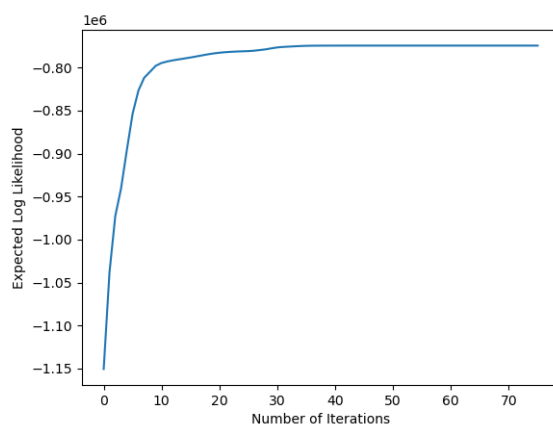


(a) Original Image



(b) Corresponding graph for the CH indices

Figure 6



We segment with 6 clusters.



(a) 55 iterations of KMeans, 75 iterations of EM, 6 clusters



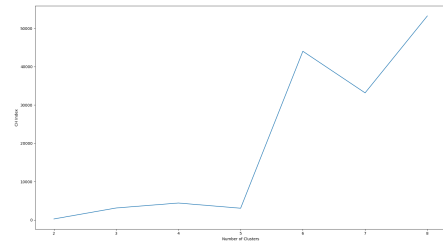
(b) Refined

Figure 7

Identification of regions and targets (i.e. vegetation, road, water etc.) in satellite images is quite essential, and image segmentation is quite a powerful tool for that.



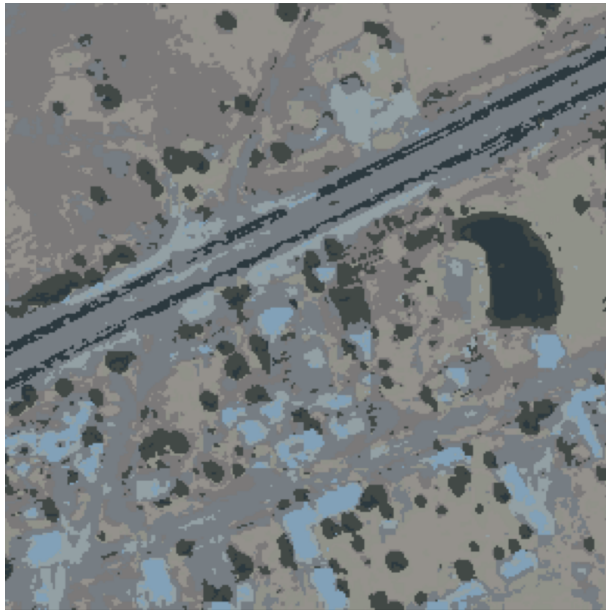
(a) Original Image



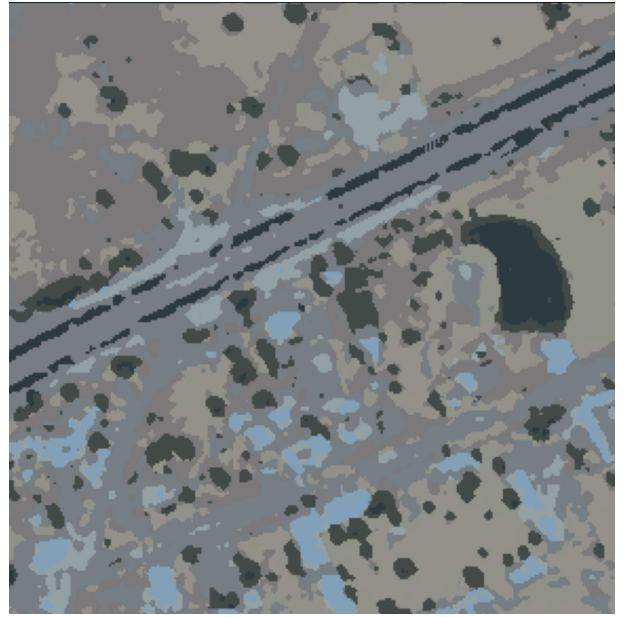
(b) Corresponding graph for the CH indices

Figure 8

We segment with 8 clusters, since satellite images usually require a slightly higher number of categories to identify different regions. The following is a segmentation.

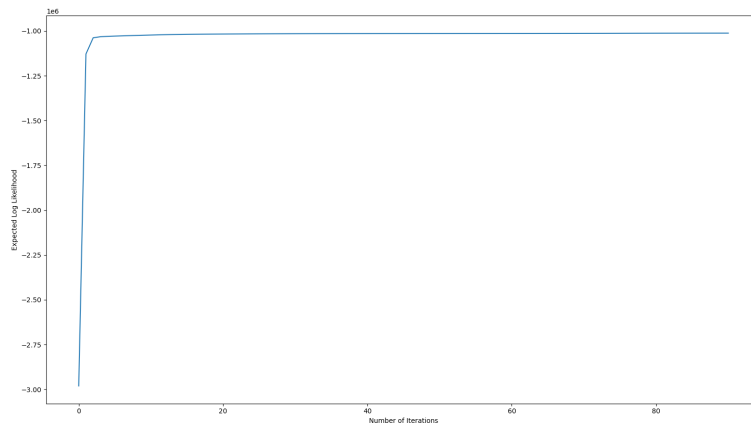


(a) 55 iterations of KMeans, 90 iterations of EM, 8 clusters



(b) Refined

Figure 9



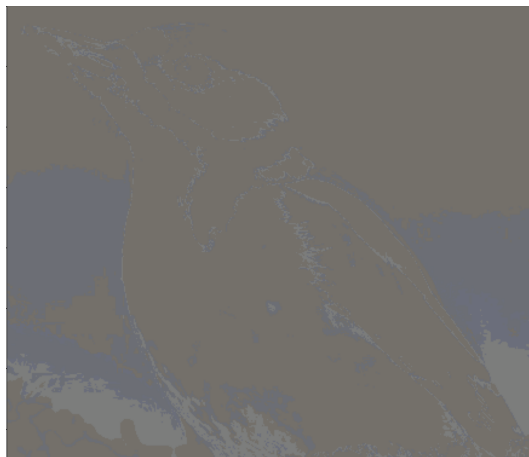
Briefly comparing the EM Algorithm And k-Means clustering:

We applied a relatively simpler algorithm to separate mixtures in cluster analysis, the popular *k*-means algorithm, where we start with a set of cluster centroids and at every iteration, assign a sample point to that centroid for which the Euclidean distance between the centroid and the point is minimised. Under certain regularity conditions, this process converges and follows a somewhat hard assignment rule (unlike the EM algorithm). We present both results for a comparative illustration of the two algorithms. There is, of course, no way to define which process performs better since the objects in an ideal setting are often unidentifiable (if they are already discernible, cluster analysis makes no sense in the first place).

The following is a comparison of two results, one obtained from KMeans and the other from fitting a Gaussian Mixture model.



(a) Original Image



(b) KMeans with 70 iterations, 4 clusters

Figure 10

Here, the segments almost seem indiscernible. Now, upon initialising using the KMeans and then segmenting using GMMs, the following is obtained.

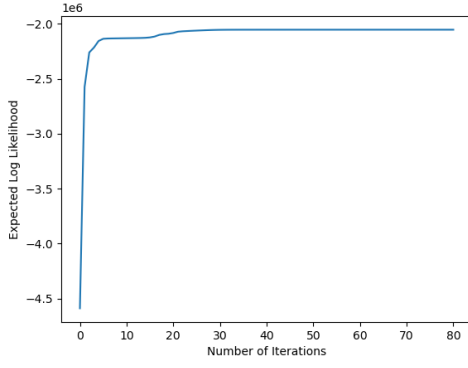


(a) 55 KMeans Iterations, 80 EM iterations, 4 clusters

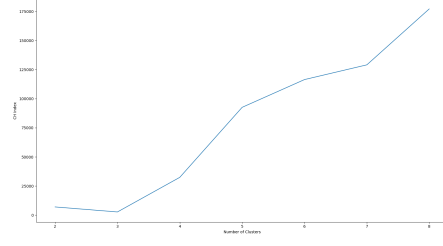


(b) Refined

Figure 11



(a) Log Likelihood Graph



(b) CH indices

Figure 12

Findings:

1. Vector Median Filtering

This was done to, in a way, smoothen the original image of possibly fine details to ensure the pixels become more feasible for clustering, i.e. the segments become somewhat more separated. A vector median filter was used for this purpose. For each pixel, a kernel of a specified window size (for a window size of k , a $k \times k$ kernel is taken) is considered centered at the pixel and the pixel is assigned the value of the median of the pixel vectors across the kernel. We chose the following measure of median for a d - dimensional dataset $\mathbf{S} = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n\}$.

$$\mathbf{x}_j = \arg \min_i \sum_{\mathbf{x} \in \mathbf{S}} \|\mathbf{x}_i - \mathbf{x}\|^2$$

We took a window size of 3. The pixels for which the corresponding kernel doesn't lie entirely within the image, we take the portion of it which lies within and take the median of those pixels.

Here is a result:

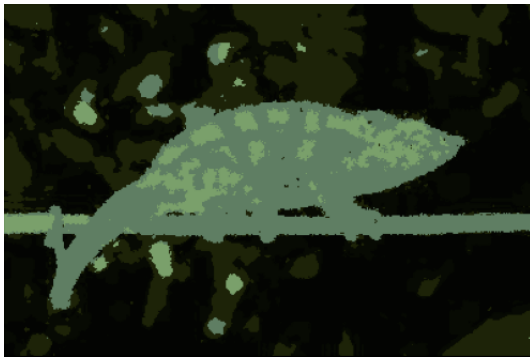


(a) Original Image

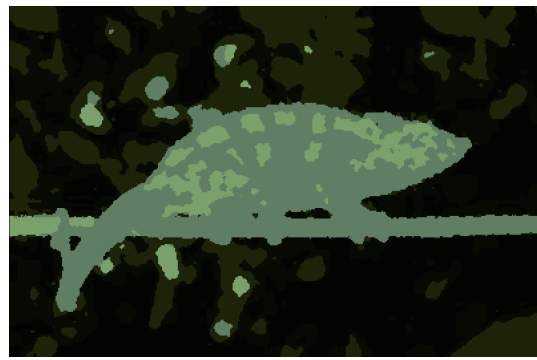


(b) Filtered, window size=3

Figure 13



(a) 50 iterations of KMeans, 70 iterations of EM,
5 clusters



(b) Refined

Figure 14

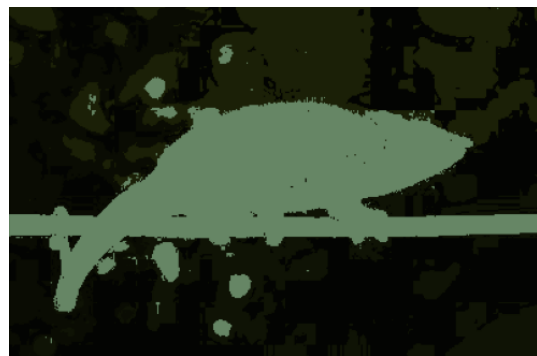
However, the results didn't differ much from those without VMF applied.

2. Applying this on a 5 dimensional feature space

Here, we considered the locations of the pixels along with the RGB colour components. However, this provided somewhat unsatisfactory results and so wasn't proceeded with. Here is a result:



(a) Original Image



(b) 55 iterations of KMeans, 80 iterations of EM,
5 clusters

Figure 15

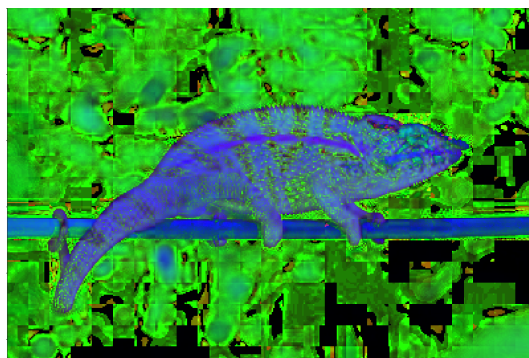
3. In HSV Colour Space

We applied this in the HSV colour space instead of RGB and the results seemed to be somewhat sensitive to noise.

The following is a result:

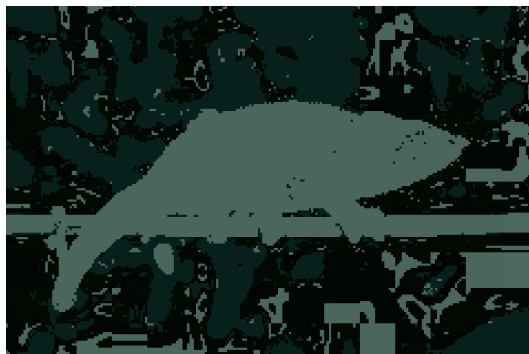


(a) Original Image



(b) In HSV color space

Figure 16



The result obtained above is after 40 iterations of KMeans and 60 iterations of EM, with 4 clusters. The output is seemingly sensitive to the background noise.

Acknowledgement

We express our gratitude to our instructor, Professor Arnab Chakraborty, for his guidance and support all through the course of this project.

This project was co-authored by
BS2312, BS2313, BS2328, BS2311, BS2316.